

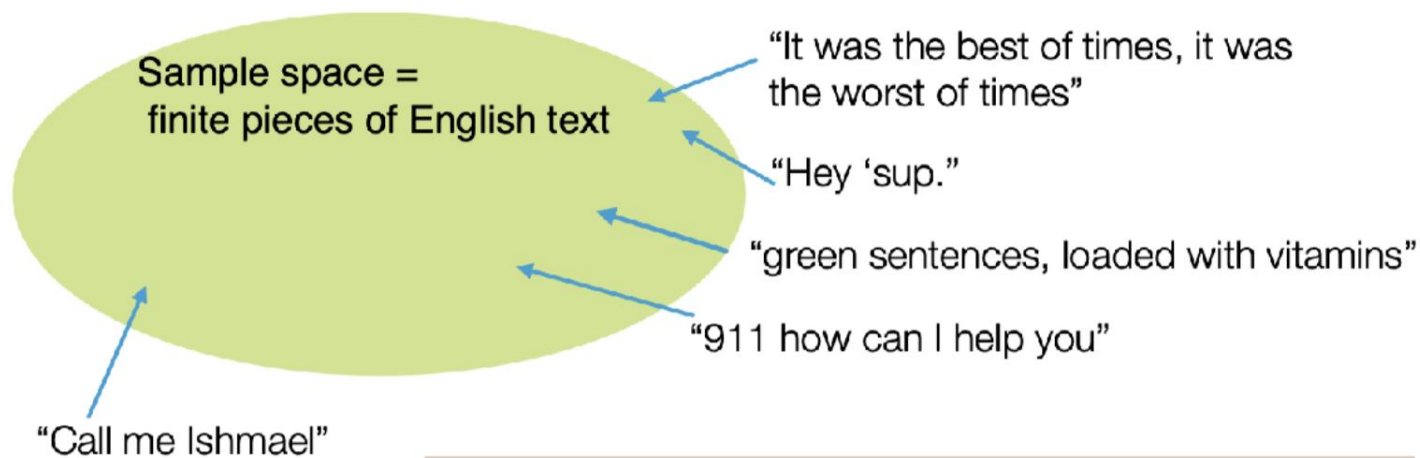
n-gram Language Models

Plan

- What is an n-gram language model?
- Generating from a language model
- Evaluating a language model (perplexity)
- Smoothing: additive, interpolation, discounting

What is a language model?

- A probabilistic model of a sequence of words
- Joint probability distribution of words $w_1, \dots, w_n$:
$$P(w_1, \dots, w_n)$$



(i.e., $\Pr[w_1 w_2 w_3 \dots w_n]$ associated with every finite word sequence $w_1 w_2 w_3 \dots w_n$ (including nonsensical ones))

How likely is a given phrase, sentence, paragraph or even a document?

Chain rule

Conditional probability:

$$p(w \mid w_1, w_2), \forall w \in V$$

$$p(w_1, w_2, w_3, \dots, w_n) = .$$

$$p(w_1)p(w_2 \mid w_1)p(w_3 \mid w_1, w_2) \times \dots \times p(w_n \mid w_1, w_2, \dots, w_{n-1})$$

➤ Sentence: “the cat sat on the mat”

$$\begin{aligned} P(\text{the cat sat on the mat}) &= P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{the cat}) \\ &\quad * P(\text{on}|\text{the cat sat}) * P(\text{the}|\text{the cat sat on}) \\ &\quad * P(\text{mat}|\text{the cat sat on the}) \end{aligned}$$

Implicit order

Language models are everywhere



Q how is the weather in new

Q how is the weather in new **york**

Q how is the weather in new **zealand**

Q how is the weather in new **orleans**

Q how is the weather in new **jersey**

Q how is the weather in new **orleans in february**

Q how is the weather in new **york in march**

Q how is the weather in new **orleans in january**

Q how is the weather in new **mexico**

Q how is the weather in new **york in february**

Q how is the weather in new **orleans in december**

Google Search I'm Feeling Lucky

Report inappropriate predictions

New Message Cancel

To:

> Language models are the

best | most | same

Estimating probabilities



$$P(\text{sat}|\text{the cat}) = \frac{\text{count}(\text{the cat sat})}{\text{count}(\text{the cat})}$$

$$P(\text{on}|\text{the cat sat}) = \frac{\text{count}(\text{the cat sat on})}{\text{count}(\text{the cat sat})}$$

⋮

Maximum
likelihood
estimate (MLE)

➤ Assume we have a vocabulary of size V , how many sequences of length n do we have?

➤ A nV

➤ B n^V

➤ C V^n

➤ D V/n

Estimating probabilities



$$P(\text{sat}|\text{the cat}) = \frac{\text{count}(\text{the cat sat})}{\text{count}(\text{the cat})}$$

$$P(\text{on}|\text{the cat sat}) = \frac{\text{count}(\text{the cat sat on})}{\text{count}(\text{the cat sat})}$$

⋮

Maximum
likelihood
estimate (MLE)

➤ With a vocabulary of size V , # sequences of length $n = V^n$

- Typical English vocabulary $\sim 40\text{k}$ words
- Even sentences of length ≤ 11 results in more than 4×10^{50} sequences. Too many to count!
- (For reference, # of atoms in the earth $\sim 10^{50}$)

Markov assumption

- Use only the recent past to predict the next word
- Reduces the number of estimated parameters in exchange for modeling capacity

- 1st order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{the})$$

- 2nd order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{on the})$$



Andrey Markov

K-th order Markov

- Consider only the last k words (or less) for context which implies the probability of a sequence is:

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

(assume $w_j = \phi \quad \forall j < 0$)

Need to estimate counts for up to (k+1) grams

n-gram models

➤ **Unigram** $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$ e.g. $P(\text{the}) P(\text{cat}) P(\text{sat})$

➤ **Bigram** $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$ e.g. $P(\text{the}) P(\text{cat} | \text{the}) P(\text{sat} | \text{cat})$

➤ and Trigram, 4-gram, and so on.

*Larger the n , more accurate and better the language model
(but also higher costs)*

Caveat: Assuming infinite data!

Estimating probabilities

› Consider the following corpus

<s> I like apples </s>

<s> You like strawberries </s>

<s> You like apples </s>

Note: <s> and </s> are starting and ending tokens

› What's the bigram probability $P(\text{apples} \mid \text{like})$?

› (A) $1/3$ (B) $2/3$ (C) $1/2$ (D) 1

Estimating probabilities

› Consider the following corpus

<s> I like apples </s>

<s> You like strawberries </s>

<s> You like apples </s>

Note: <s> and </s> are starting and ending tokens

› What's the bigram probability $P(\text{apples} \mid \text{like})$?

› (A) $1/3$ (B) $2/3$ (C) $1/2$ (D) 1

$$P(\text{apples} \mid \text{like}) = \frac{\text{Count}(\text{"like apples"})}{\text{Count}(\text{"like"})} = \frac{2}{3}$$

Estimating probabilities

➤ Consider the following corpus

<s> I like apples </s>

<s> You like strawberries </s>

<s> You like apples </s>

Note: <s> and </s> are starting and ending tokens

➤ Using the bigram model, what's the probability of the sentence "<s> I like strawberries </s>"? Ignore the probability of <s>.

➤ (A) $\frac{4}{9}$ (B) $\frac{1}{3}$ (C) $\frac{2}{9}$ (D) $\frac{1}{9}$

Estimating probabilities

> Consider the following corpus

<s> I like apples </s>

<s> You like strawberries </s>

<s> You like apples </s>

Note: <s> and </s> are starting and ending tokens

> Using the bigram model, what's the probability of the sentence "<s> I like strawberries </s>"? Ignore the probability of <s>.

> (A) 4/9 (B) 1/3 (C) 2/9 (D) 1/9

$$P(\text{<s> I like strawberries </s>}) = \frac{1}{3} \cdot 1 \cdot \frac{1}{3} \cdot 1$$

Generating from a language model

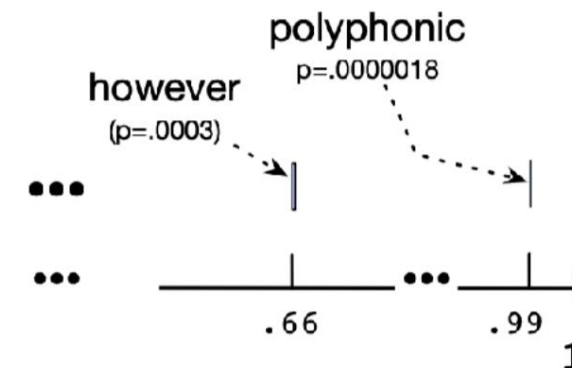
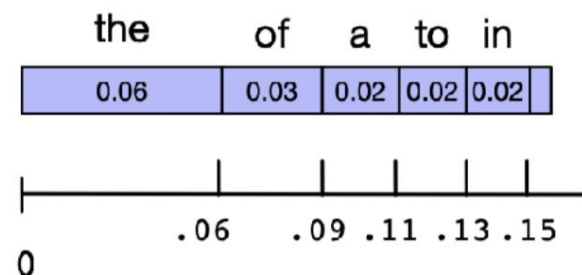
➤ Given a language model, how to generate a sequence?

$$\text{Bigram } P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$$

➤ Generate the first word $w_1 \sim P(w)$

➤ Generate the second word $w_2 \sim P(w | w_1)$

➤ ...



Generations

› Unigram

- › To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

› Bigram

- › Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
- › What means, sir. I confess she? then all sorts, he is trim, captain.

› Trigram

- › Fly, and will rid me these news of price. Therefore the sadness of parting, as they say,
- › This shall forbid it should be branded, if renown made it empty

Generations

➤ Unigram

➤ Bigram

➤ Trigram

Typical LMs are not sufficient to handle long-range dependencies

“The **woman/man** could not go to work that day
because **she/he** had a doctor’s appointment”

Generations

Example from a GPT-2 output (2019): **prompt aka. conditional context**

With the start of the new academic year, Princeton has an opportunity to help provide a new generation of women with a diverse set of academic resources for higher education. We are offering the resources of the Princeton-McGill program specifically to women with undergraduate degrees who would like to enhance their academic experience. Princeton-McGill offers a comprehensive suite of services for women and their families including a variety of graduate programs, support programs, and the opportunity to serve as leaders in their communities with a wide variety of programs, activities and services. For the upcoming fall, Princeton-McGill will also offer its Women's Center , which is located in a renovated women's dorm. At Princeton, we are working with the Princeton-McGill community to develop a suite of programs that are designed to give new and returning students a strong foundation for a successful, rewarding graduate career. The Women's Center , the Princeton-McGill Women's Center provides a range of supports to address the specific needs of female doctoral degree graduates. Programs are tailored to meet the unique needs of women under the age of 28, women and families

<https://talktotransformer.com/>

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_{i-1024}, \dots, w_{i-2}, w_{i-1})$$

Modern LMs can handle much longer contexts!

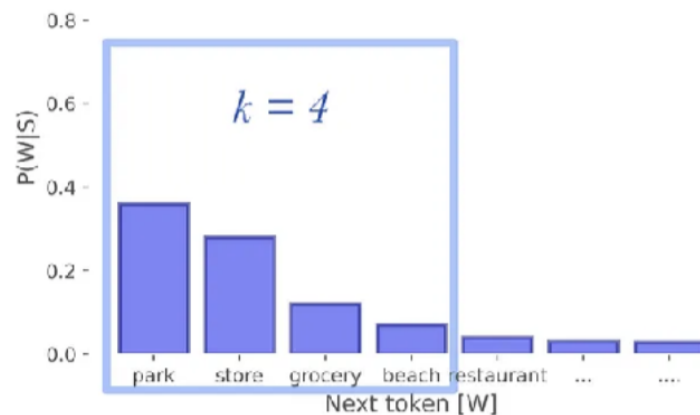
Generation methods (advanced)

➤ Greedy: choose the most likely word!

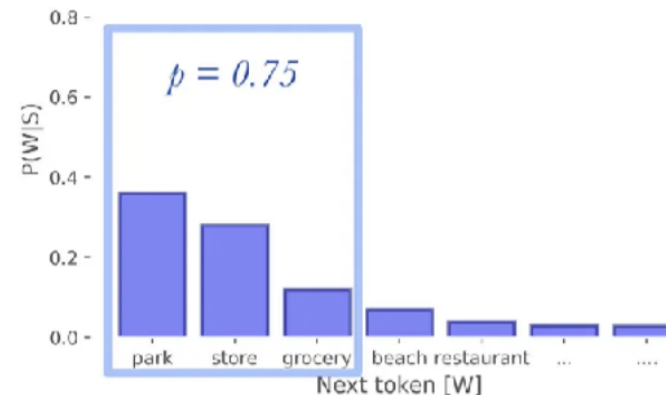
➤ To predict the next word given a context of two words w_1, w_2 :

$$w_3 = \arg \max_{w \in V} P(w | w_1, w_2)$$

➤ Top-k vs top-p sampling: “The boy went to the _____”



Top-k sampling

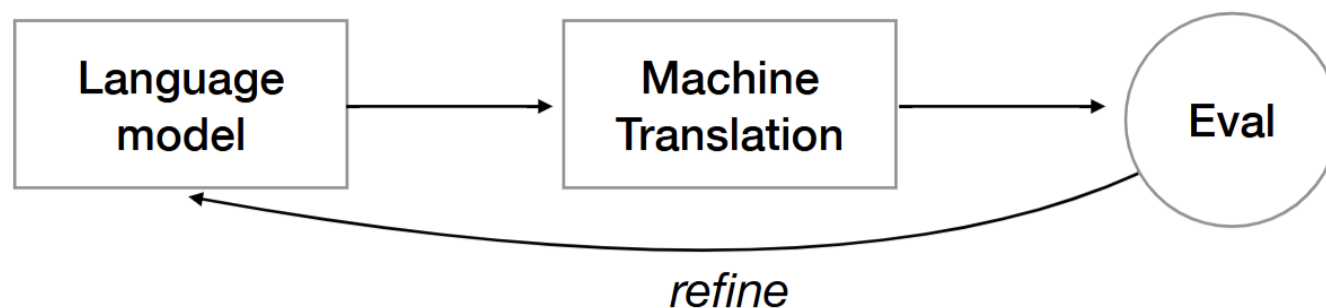


Top-p sampling

Evaluating a language model

➤ Extrinsic evaluation

- Train LM → apply to task → observe accuracy
- Directly optimized for downstream applications
 - *higher task accuracy → better model*
- Expensive, time consuming
- Hard to optimize downstream objective (indirect feedback)



Intrinsic evaluation of language models

- Train parameters on a suitable training corpus
 - Assumption: observed sentences $\sim$ good sentences
- Test on different, unseen corpus
 - If a language model assigns a higher probability to the test set, it is better
- Evaluation metric - perplexity!

Motivation: Shannon game

- Goal for language models: model $P(w_1, \dots, w_k)$ or $P(w_k | w_1, \dots, w_{k-1})$ well
- Shannon game: How well can we predict the next word?
 - I always order pizza with cheese and _____
 - The 33rd president of the US was _____
 - I saw a _____

Motivation: Shannon game

- Goal for language models: model $P(w_1, \dots, w_k)$ or $P(w_k | w_1, \dots, w_{k-1})$ well
- Shannon game: How well can we predict the next word?
 - I always order pizza with cheese and _____
 - The 33rd president of the US was _____
 - I saw a _____

{ mushrooms 0.1
pepperoni 0.1
anchovies 0.01
...
friend rice 0.0001
...

How would a unigram model do?

Motivation: Shannon game

- Goal for language models: model $P(w_1, \dots, w_k)$ or $P(w_k | w_1, \dots, w_{k-1})$ well
- Shannon game: How well can we predict the next word?
 - I always order pizza with cheese and _____
 - The 33rd president of the US was _____
 - I saw a _____

{ mushrooms 0.1
pepperoni 0.1
anchovies 0.01
...
friend rice 0.0001
...

How would a unigram model do?

Not well, it would assign the most probability to the most common word (i.e. the word that occurs the most in the training corpus).

Better language model = assigns higher probability to word that actually occurs.

Perplexity (ppl)

- Measure of how well a LM predicts the next word
- For a test corpus with words $w_1, \dots, w_n$

$$\text{Perplexity} = P(w_1, w_2, \dots, w_n)^{-1/n}$$

- $ppl(S) = 2^x, x = -\frac{1}{n} \sum_{i=1}^n \log_2 P(w_i | w_1, \dots, w_{i-1})$
- Unigram model
- Minimizing perplexity $\sim$ maximizing probability of corpus

Intuition on perplexity

➤ $ppl(S) = 2^x, x = -\frac{1}{n} \sum_{i=1}^n \log_2 P(w_i | w_1, \dots, w_{i-1})$

➤ If our k-gram model (with vocabulary V) has following probability:

$$P(w | w_{i-k}, \dots, w_{i-1}) = \frac{1}{|V|}, \quad \forall w \in V$$

what is the perplexity of the test corpus?

➤ A) $2^{|V|}$ B) $|V|$ C) $|V|^2$ D) $2^{-|V|}$

Intuition on perplexity

➤ $ppl(S) = 2^x, x = -\frac{1}{n} \sum_{i=1}^n \log_2 P(w_i | w_1, \dots, w_{i-1})$

➤ If our k-gram model (with vocabulary V) has following probability:

$$P(w | w_{i-k}, \dots, w_{i-1}) = \frac{1}{|V|}, \quad \forall w \in V$$

what is the perplexity of the test corpus?

➤ A) $2^{|V|}$ B) $|V|$ C) $|V|^2$ D) $2^{-|V|}$

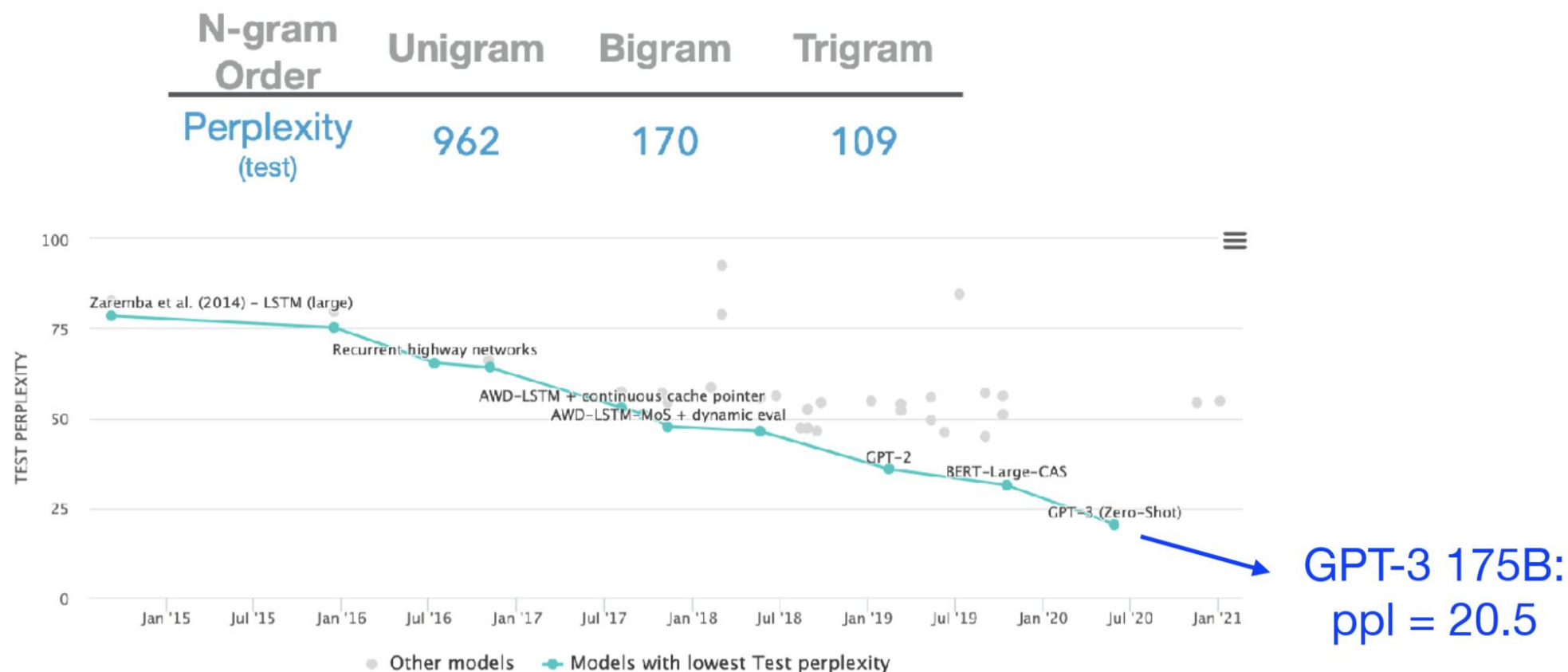
$$ppl = 2^{-\frac{1}{n} n \log(1/|V|)} = |V|$$

Measure of model's uncertainty about next word (aka 'average branching factor')

branching factor = # of possible words following any word

Perplexity

Training corpus 38 million words, test corpus 1.5 million words, both **WSJ**



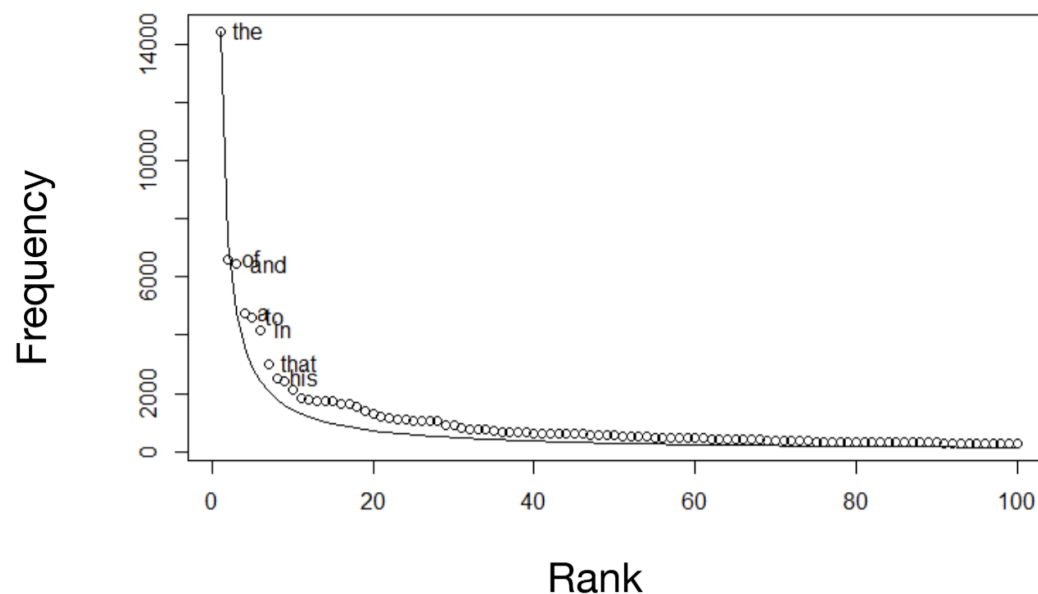
<https://paperswithcode.com/sota/language-modelling-on-penn-treebank-word>

Generalization of n-grams

- Not all n-grams in the test set will be observed in training data
- Test corpus might have some that have zero probability under our model
 - Training set: Google news
 - Test set: Shakespeare
 - $P(\text{affray} \mid \text{voice doth us}) = 0 \Rightarrow P(\text{test corpus}) = 0$
 - Perplexity is not defined.

$$\text{ppl}(S) = 2^x \quad \text{where} \\ x = -\frac{1}{n} \sum_{i=1}^n \log P(w_i \mid w_1 \dots w_{i-1})$$

Sparsity in language



$$freq \propto \frac{1}{rank}$$

Zipf's Law

- Long tail of infrequent words
- Most finite-size corpora will have this problem.

Smoothing

- **Handle sparsity by making sure all probabilities are non-zero in our model**
 - **Additive: Add a small amount to all probabilities**
 - **Interpolation: Use a combination of different granularities of n-grams**
 - **Discounting: Redistribute probability mass from observed n-grams to unobserved ones**

Smoothing intuition

When we have sparse statistics:

$P(w \mid \text{denied the})$

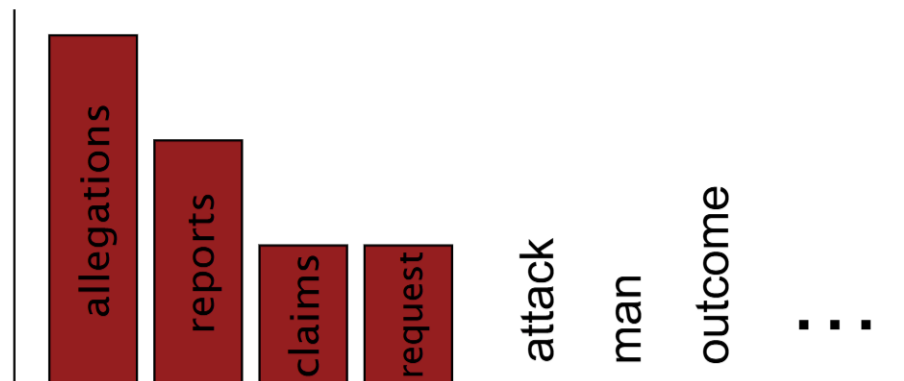
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

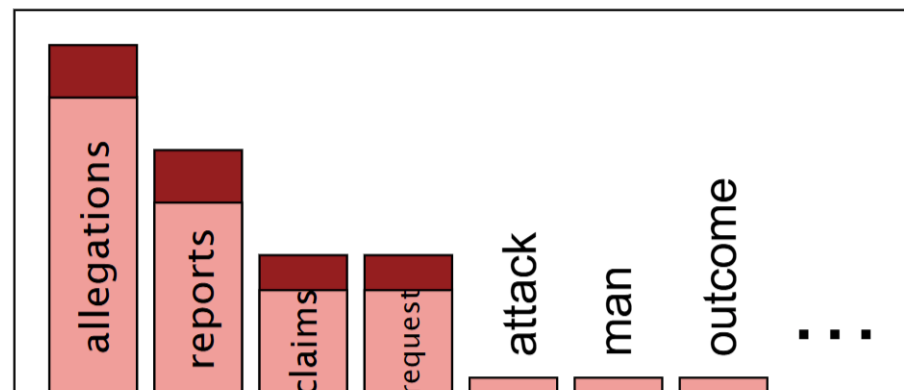
1.5 reports

0.5 claims

0.5 request

2 other

7 total



(Slide credit: Dan Klein)

Laplace smoothing

- Also known as add-alpha
- Simplest form of smoothing: Just add α to all counts and renormalize!
- Max likelihood estimate for bigrams:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

- After smoothing:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha |V|}$$

Raw bigram counts (Berkeley restaurant corpus)

➤ Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Add 1 to all the entries in the matrix

(Slide credit: Dan Jurafsky)

Smoothed bigram probabilities

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha|V|} \quad \alpha = 1$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Linear Interpolation

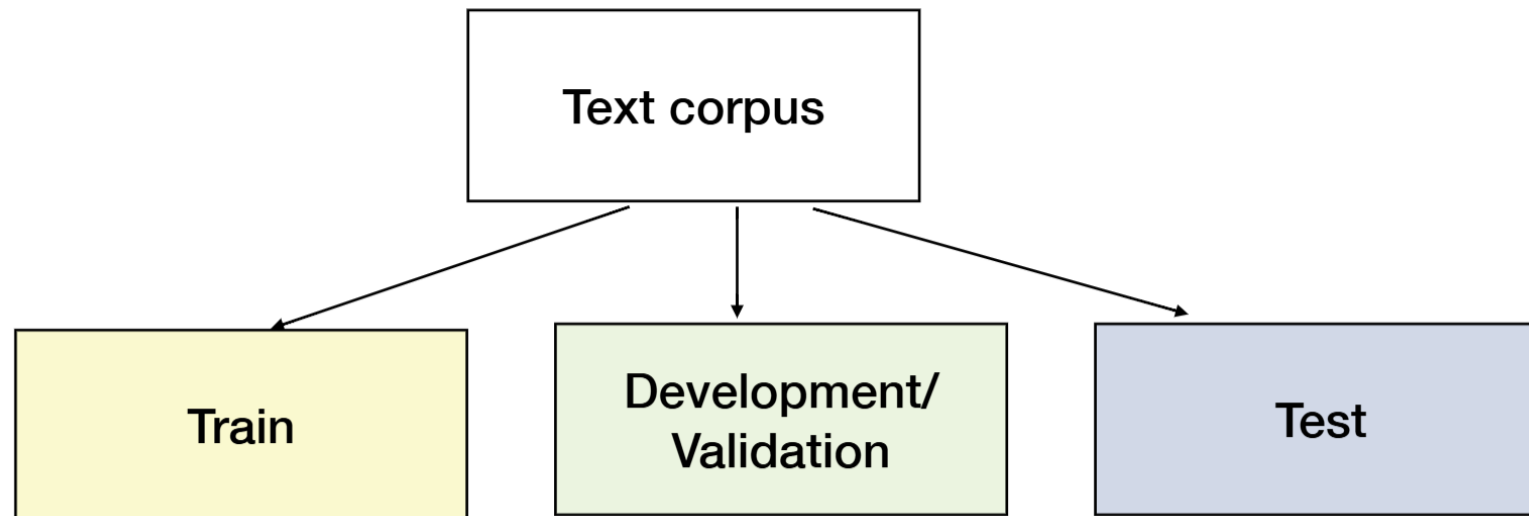
$$\begin{aligned}\hat{P}(w_i \mid w_{i-2}, w_{i-1}) = & \lambda_1 P(w_i \mid w_{i-2}, w_{i-1}) && \text{Trigram} \\ & + \lambda_2 P(w_i \mid w_{i-1}) && \text{Bigram} \\ & + \lambda_3 P(w_i) && \text{Unigram}\end{aligned}$$

$$\sum_i \lambda_i = 1$$

- Use a combination of models to estimate probability
- Strong empirical performance

How can we choose lambdas?

- First, estimate n-gram prob. on training set
- Then, estimate lambdas (hyperparameters) to maximize probability on the held-out development/validation set
- Use best model from above to evaluate on test set



Discounting

- Determine some “mass” to remove from probability estimates
- More explicit method for redistributing mass among unseen n-grams
- Just choose an absolute value to discount (usually <1)

Absolute Discounting

➤ Define $\text{Count}^*(x) = \text{Count}(x) - 0.5$

➤ Missing probability mass:

$$\alpha(w_{i-1}) = 1 - \sum_w \frac{\text{Count}^*(w_{i-1}, w)}{\text{Count}(w_{i-1})}$$

$$\alpha(\text{the}) = 1 - \frac{43}{48} = 5/48$$

➤ Divide this mass between words w for which $\text{Count}(\text{the}, w) = 0$

x	$\text{Count}(x)$	$\text{Count}^*(x)$	$\frac{\text{Count}^*(x)}{\text{Count}(x)}$
the	48		
the, dog	15	14.5	14.5/48
the, woman	11	10.5	10.5/48
the, man	10	9.5	9.5/48
the, park	5	4.5	4.5/48
the, job	2	1.5	1.5/48
the, telescope	1	0.5	0.5/48
the, manual	1	0.5	0.5/48
the, afternoon	1	0.5	0.5/48
the, country	1	0.5	0.5/48
the, street	1	0.5	0.5/48

Absolute Discounting

x	$\text{Count}(x)$	$\text{Count}^*(x)$	$\frac{\text{Count}^*(x)}{\text{Count}(x)}$
the	48		
the, dog	15	14.5	14.5/48
the, woman	11	10.5	10.5/48
the, man	10	9.5	9.5/48
the, park	5	4.5	4.5/48
the, job	2	1.5	1.5/48
the, telescope	1	0.5	0.5/48
the, manual	1	0.5	0.5/48
the, afternoon	1	0.5	0.5/48
the, country	1	0.5	0.5/48
the, street	1	0.5	0.5/48

$$\alpha(\text{the}) = 10 \times 0.5/48 = 5/48$$

$$P_{\text{abs_discount}}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} \quad \text{if } c(w_{i-1}, w_i) > 0$$

$$\alpha(w_{i-1}) \frac{P(w_i)}{\sum_{w' | c(w_{i-1}, w')=0} P(w')} \quad \text{if } c(w_{i-1}, w_i) = 0$$

Unigram probabilities

Quick Question

- Suppose we are training a trigram (3-gram) language model on the following short corpus:

“I love machine learning. I love deep learning.”

- What is the probability $P(\text{learning}|\text{love deep})$ under the maximum likelihood estimation (MLE)?
- If add-one (Laplace) smoothing is applied, how would the probability $P(\text{learning}|\text{love deep})$ change? Assume the vocabulary size is 6: {I, love, machine, deep, learning, .}.